COMPLETE — MASTER BIBLE + TRACKS 1-4

Senior / Lead Java Interview Prep

The complete system in one file — the Master Bible plus all four tracks, in reading order. Use the PDF bookmarks / outline panel to jump straight to any section.

· Contents

#	SECTION	WHAT IT IS
0	**Master Bible**	Operating system and scorecard: philosophy, readiness model, story bank, emergency questions, gold-standard answers, pre-interview cheat sheet.
1	**Track 1 — Near-Term Execution**	30-day daily plan plus a ~195-problem core-Java practice bank (plain-loop, no built-ins).
2	**Track 2 — Senior / Tech Lead Mastery**	12-week production-readiness and leadership build, project mapping, and debugging scenarios.
3	**Track 3 — Java + AI/RAG Differentiator**	6-month roadmap to position Java + AI/RAG honestly, with a portfolio checklist.
4	**Track 4 — Future Reference / Advanced Gaps**	Conditional, learn-only-if-a-JD-asks roadmap for deep DSA, JVM, Kubernetes, AWS, security.

> **WHERE TO START**
>
> Read the **Master Bible** first — it frames the whole system and is your scorecard — then jump to the track that matches your timeline. The core principle throughout: **output beats input**.

SENIOR / LEAD JAVA ENGINEER · OPERATING SYSTEM

Java Interview Master Bible

1 The Five-PDF System

PDF	USE IT FOR
Master Bible	Your operating system and scorecard. Philosophy, readiness, story bank, emergency questions, cheat sheet.
Track 1	Daily execution when interviews are near — coding, Spring, Kafka, SQL, microservices, design, projects.
Track 2	Sounding like a true senior / tech lead — design quality, production readiness, leadership.
Track 3	Java + AI/RAG + architecture positioning for stronger roles and rates.
Track 4	Future reference only — deeper algorithm, cloud, frontend, JVM or platform depth when a JD demands it.

2 The Operating Principle — Output Beats Input

This is the spine of the whole system. Interviews are performances, not exams. What you can **say out loud, under time, to another person** is the only thing scored — not how many videos you watched or pages you read.

- ☐ **Speak two answers out loud every day**, timed. Reading the PDFs is not the goal.
- ☐ **Get a real mock with a human early** — by the end of week 1, not week 4. Self-mocks are graded too kindly.
- ☐ **Convert every abstract answer into a real WSI / CARB / payments example.** That is what makes deep experience believable.
- ☐ **Reading and video are support, never the work.** Video is optional — commute or treadmill only.

> **RULE OF THUMB**
>
> If you only have 30 minutes, spend it **speaking and solving**, not watching. Solve, speak, explain the trade-off, connect to a project, handle the follow-up.

3 The Readiness Model

Where the offer actually lives. Spring + Kafka + project deep-dive together are over half the weight — do not over-invest in algorithms at their expense.

SKILL	WEIGHT	TARGET BEHAVIOR
Spring / JPA / Security	20%	DI, beans, transactions, N+1, JWT/OAuth/OIDC — clearly, with project examples.
Kafka / Microservices	17%	Delivery semantics, idempotency, retries, DLQ, downstream-failure handling, saga/ outbox.
Project deep dive	15%	4–5 airtight WSI/CARB/payments stories with role, decision, trade-off, honest impact.
Coding filters	14%	Solve common Java problems in 15–25 min and narrate the approach.
System design	12%	Design eCommerce and common distributed systems with a repeatable structure.
SQL / DB design	11%	Joins/window queries; indexing, tuning, RDBMS vs NoSQL, sharding.
Behavioral / lead	8%	Speak like an owner: mentoring, reviews, conflict, incidents, delivery, decisions.
AI/RAG differentiator	3%	A bonus edge, framed honestly — never an inflated production claim.

4 Weekly Operating Schedule

DAY	MAIN FOCUS	MINIMUM OUTPUT
Monday	Coding + Java	2 coding problems + 1 Java concept spoken.
Tuesday	Coding + Spring	1 coding problem + 2 Spring answers spoken.
Wednesday	Kafka + Microservices	2 Kafka/microservice answers + 1 WSI anchor.
Thursday	SQL + System Design	2 SQL queries + 1 design walkthrough.
Friday	Project Deep Dive + Behavioral	1 WSI story + 1 CARB/payments story + 1 lead story.
Saturday	Mock + Track 2	Timed mock + SOLID/12-factor/patterns/production readiness.
Sunday	Mock review + Track 3	Review misses + AI/RAG or portfolio progress.

5 Interview-Week Emergency Mode

When an interview lands within ~7–10 days, switch modes — and do **not** feel guilty. The system expects this.

- ☐ **Drop Track 3 completely** for the week.
- ☐ **Keep one 20-minute Track 2 touch** — one senior concept + one project example — so depth never hits zero.
- ☐ **Every day:** one coding problem, two spoken technical answers, one project story.
- ☐ **Backend-heavy interview?** Prioritize Spring, Kafka, SQL, microservices and project deep-dive over extra algorithms.
- ☐ **Day before:** no new topics. Review your strongest stories, the cheat sheet, and the JD stack.

6 The Spaced Review Loop — Beat the Forgetting Curve

Linear study means that by Day 26 the Day 5 LRU details have faded — and a senior screen can hit anything. Force recall cheaply:

- ☐ **Morning recall (5 min):** yesterday's topic + one random past topic, from memory, before any new material.
- ☐ **Every mock pulls one random older topic** alongside the new ones.
- ☐ **Weekly:** re-touch your two weakest areas from the dashboard.

7 The Golden Answer Formula

STEP	WHAT TO SAY
1. Direct answer	One clear line, before any explanation.
2. Why it matters	The business or technical risk / benefit.
3. How it works	The mechanism or flow.
4. Trade-offs	Limits, edge cases, alternatives.
5. Real example	WSI, CARB, or payments.
6. What you'd improve	Shows seniority and reflection.

8 Gold-Standard Answer Skeletons

The formula applied to four questions you will almost certainly be asked. Learn the shape, then swap in your own numbers.

GOLD **"How do you handle a downstream service failure?"**

DIRECT	Timeout + retry with backoff + circuit breaker + fallback — and idempotency so the retries are safe.
WHY	A slow or dead dependency otherwise cascades and exhausts your threads.
HOW	Resilience4j: timeout caps the wait, retry handles blips, the breaker opens on sustained failure and serves a fallback.
TRADE-OFF	A fallback can be stale; retries without idempotency create duplicates.
EXAMPLE	The Accertify fraud call in WSI checkout — protect checkout latency when risk evaluation is slow.
IMPROVE	Add bulkhead isolation so one slow dependency can't starve the rest.

GOLD **"How do you handle duplicate Kafka messages?"**

DIRECT	An idempotent consumer keyed on event ID or business key.
WHY	Real systems run at-least-once, so the same event will sometimes arrive twice.
HOW	Before acting, check a processed-ID store; if seen, skip; otherwise process and record the ID.
TRADE-OFF	The dedup store costs storage and a lookup; size its TTL to the redelivery window.
EXAMPLE	The gift-card balance-inquiry risk event — no duplicate downstream calls or wrong status.
IMPROVE	Use the DB transaction + an outbox so processing and the dedup record commit together.

GOLD **"Explain the JPA N+1 problem and how you fix it."**

DIRECT	One query loads the parents, then one extra query per child — N+1 round trips.
WHY	It quietly turns one logical read into hundreds, blowing up latency under load.
HOW	A fetch join or `@EntityGraph` loads children in one query; batch fetching is the middle ground.
TRADE-OFF	Multiple fetch joins cause a cartesian explosion — prefer EntityGraph or batch size for several collections.
EXAMPLE	Loading a checkout order with its line items and promotions in one go.
IMPROVE	Read paths often want projection DTOs instead of full entities.

GOLD **“How do you guarantee idempotency in checkout/payment?”**

DIRECT	A client-supplied idempotency key (or business key) that makes a repeated request a no-op.
WHY	Network retries and double-clicks must never charge twice or place two orders.
HOW	Persist the key with the result; on a repeat, return the stored result instead of re-executing.
TRADE-OFF	You need a key store and a defined retention window; concurrent repeats need a unique constraint.
EXAMPLE	Idempotent checkout/payment endpoints at WSI; in payments, the transaction reference is the key.
IMPROVE	Combine with an outbox so the side effect and the key commit atomically.

GOLD **“API latency jumped from 200ms to 5s — how do you debug?”**

DIRECT	Walk a diagnostic ladder — confirm the symptom, read metrics first, then narrow to a layer before touching code.
WHY	Latency can come from app code, GC pauses, thread-pool exhaustion, the DB, a slow downstream, or the network — guessing wastes the outage.
HOW	Check dashboards (p95/p99, error rate, CPU, heap, GC, pool saturation); narrow with traces and correlation IDs; confirm with thread / heap dumps or slow-query logs.
TRADE-OFF	A fast rollback restores service but hides the root cause — capture diagnostics first if you can.
EXAMPLE	Checkout p95 regression at WSI traced through correlation IDs to a slow Accertify call; fixed with timeout + circuit breaker.
IMPROVE	Add an alert and a runbook so the same regression is caught automatically next time.

9 The Story Bank — Your Crown Jewel

The asset no one else can copy, and the part most candidates do worst. Do not leave these as titles — write each out and rehearse to a tight two minutes. Aim for **4–5 airtight stories, not 15 shallow ones**: if one gets drilled and it is thin, the whole project-anchoring strategy backfires.

STAR-plus template (every story)

- ☐ **Context** — system, scale, your team, the stakes (1–2 lines).
- ☐ **Problem** — what was actually wrong or needed.
- ☐ **Your role** — specifically what *you* did, not “we”.
- ☐ **Decision + alternatives** — the call you made and what you rejected, and why.
- ☐ **Impact** — a defensible, bounded number.
- ☐ **What you’d do differently** — seniority and reflection.

> **HONESTY RULE ON NUMBERS**
>
> No exact figures? Don’t invent precise ones — a sharp interviewer asks “how did you measure that?” Use bounded phrasing: “cut checkout p95 from roughly X to Y,” “eliminated duplicate downstream calls with an idempotent consumer.” Bounded and honest survives follow-ups; fabricated precision collapses.

Stories to build

WSI / ECOMMERCE

- ☐ Cart & checkout optimization
- ☐ Accertify fraud/risk integration
- ☐ Kafka producer/consumer (keying, group, idempotency, DLQ)
- ☐ Gift-card balance-inquiry risk event
- ☐ Tech Anchor role (reviews, offshore, decisions)

CARB · PAYMENTS

- ☐ Struts → Spring Boot migration (strangler)
- ☐ Monolith → microservices (boundaries, ownership)
- ☐ AWS migration (API GW, Cognito, S3, CloudWatch)
- ☐ Cognito / OAuth / JWT auth flow
- ☐ IMPS / payment switch (ISO8583 flow)

- [ ] Production debugging (correlation ID → root cause → fix)
- [ ] Idempotency in payments (business key, safe reprocess)

10 Top Emergency Questions

The cross-cutting set most likely to appear. Drill them as spoken answers, not silent reading.

1 Tell me about yourself for a Senior/Lead Java role.
2 Walk me through your cart/checkout work.
3 Explain the Accertify fraud/risk integration.
4 Explain a Kafka producer/consumer integration you built.
5 How do you handle duplicate Kafka messages?
6 Kafka delivery semantics: at-most / at-least / exactly-once.
7 What is consumer lag and how do you reduce it?
8 How does Kafka achieve high throughput?
9 What happens when a Kafka broker fails?
10 Kafka vs RabbitMQ?
11 How do you replay historical events safely?
12 How do you handle a downstream service failure?
13 Circuit breaker vs retry vs timeout vs bulkhead.
14 Saga vs outbox pattern.
15 Saga choreography vs orchestration.
16 REST vs gRPC in microservices.
17 Idempotency in checkout / payment / event flows.
18 Spring DI — why constructor injection?
19 Bean lifecycle and bean scopes.
20 @Bean vs @Component.
21 Spring Boot auto-configuration.
22 @Transactional propagation and isolation.
23 Why @Transactional fails on self-invocation.
24 JPA N+1 problem and fixes.
25 Lazy vs eager loading.
26 Optimistic vs pessimistic locking.
27 JWT flow.
28 OAuth2 vs OIDC.
29 Spring Security filter chain.
30 REST status codes and error design.
31 Global exception handling.
32 Request validation with @Valid.
33 SQL: second / Nth highest salary.
34 Max salary by department.
35 Window functions: ROW_NUMBER / RANK / DENSE_RANK.
36 Indexing and query tuning.
37 RDBMS vs NoSQL.
38 How do you scale a relational DB?
39 Cassandra vs PostgreSQL.
40 Design a shopping cart.
41 Design a checkout service.
42 Design gift-card balance inquiry.
43 Design fraud/risk evaluation.
44 Design a rate limiter.
45 Design a notification system.
46 Design a URL shortener.
47 Implement an LRU cache.
48 First non-repeating character.
49 Longest substring without repeating characters.
50 Two sum / move zeroes / binary search.
51 Validate a BST.
52 Kth largest element.
53 Odd/even with two threads.
54 Producer-consumer with BlockingQueue.
55 HashMap vs ConcurrentHashMap internals.
56 equals and hashCode contract.
57 Comparable vs Comparator.
58 ArrayList vs LinkedList.
59 HashSet vs TreeSet.
60 Fail-fast vs fail-safe iterators.
61 synchronized vs ReentrantLock.
62 CAS (compare-and-swap).
63 Modern Java 21/25: structured concurrency, sequenced collections, stream gatherers.
64 SOLID with project examples.
65 12-factor app in Spring Boot.
66 Design patterns used in your projects.
67 Clean architecture / controller-service-repository.
68 Observability: logs, metrics, traces, correlation ID.
69 CI/CD flow and rollback.
70 Readiness vs liveness probes.
71 Security best practices and OWASP.
72 A production-incident story.
73 A mentoring / code-review story.
74 A conflict-with-architect/product story.
75 Why should we hire you? Why this role?
76 What would you improve in your current system?
77 Your AI/RAG positioning — and questions to ask the interviewer.

11 Coverage Map Across Tracks

AREA	TRACK	READINESS BAR
Coding screens	Track 1	Common questions in 15–25 min: brute force, optimized, edge cases, complexity.
Spring / JPA / Security	Track 1 + 2	Core questions in 2–3 min, tied to WSI/CARB implementation.
Kafka / Microservices	Track 1 + 2	Producer/consumer, delivery semantics, retries, DLQ, idempotency, failure handling.
SQL / DB scaling	Track 1 + 4	Query patterns; indexing, replication, sharding, NoSQL trade-offs.
System design	Track 1 + 4	Cart, checkout, rate limiter, notification, URL shortener, feed — with trade-offs.
Tech-lead depth	Track 2	SOLID, 12-factor, patterns, clean architecture, observability, testing, CI/CD, security.
AI/RAG differentiator	Track 3	Position Spring AI/RAG honestly; build small finished proof.
Advanced gaps	Track 4	Roadmap for DP/graphs, deep React, cloud, K8s, JVM, SRE, distributed systems.

12 Readiness Dashboard

Score by *evidence*, not vibes. Fill the score only when the evidence is true.

AREA	SCORE 1–5	EVIDENCE REQUIRED
Coding	____	3 random problems solved under 25 min.
Spring / JPA / Security	____	10 spoken answers without notes.
Kafka / Microservices	____	10 spoken answers + one WSI anchor each.
SQL / DB	____	10 queries written by hand.
System design	____	2 designs out loud, end to end.
Project stories	____	4 stories in STAR-plus, 2 min each.
Behavioral / lead	____	5 lead stories with decision and lesson.
Cloud / DevOps	____	Deployment, logs, rollback explained.
React / frontend	____	Hooks, state, API call, performance basics.
AI/RAG	____	2-min honest positioning + one demo idea.

13 Mock Scorecard

The real gauge. Score each 1–5 after every mock and watch the trend. You are ready when you are consistently 4–5 on questions you have *not* pre-seen.

DATE	TOPIC	OPEN	STRUCTURE	EXAMPLE	FOLLOW-UP	TIME	SCORE
		1–5	1–5	1–5	1–5	Y/N	____
		1–5	1–5	1–5	1–5	Y/N	____
		1–5	1–5	1–5	1–5	Y/N	____
		1–5	1–5	1–5	1–5	Y/N	____
		1–5	1–5	1–5	1–5	Y/N	____

Open = answered before explaining · Structure = no rambling · Example = real project + number · Follow-up = handled the curveball · Time = finished in the window.

14 Pre-Interview Cheat Sheet — Traps & One-Liners

The day-before page. These are the distinctions that separate a senior answer from a mid-level one.

CODING

Override **equals() and hashCode() together** — never one alone.

Overflow-safe mid: **low + (high−low)/2**.

Validate BST with a **min/max range**, not just parent comparison; inorder of a BST is sorted.

Top-K → **heap of size K**, O(n log k); min-heap for k-largest.

Use a **dummy node** for any list head-modifying op.

volatile = visibility, not atomicity.

SPRING / JPA

Constructor injection — **final, testable, fail-fast**.

@Transactional is **proxy-based** — self-invocation and private methods skip it.

N+1 → **fetch join / @EntityGraph**; batch for many collections.

@Bean = explicit/3rd-party; @Component = classpath-scanned.

KAFKA

Ordering holds **only within a partition**; the key picks the partition.

Throughput = **sequential IO + zero-copy + batching + page cache**.

Design for **at-least-once + idempotent consumers**.

Exactly-once is real **inside** Kafka (idempotent producer + transactions); **end-to-end across external systems it is at-least-once + dedup**.

Broker fails → an **in-sync replica** is elected leader; `acks=all` avoids loss.

Poison messages → DLQ after bounded retries.

MICROSERVICES

Downstream failure = **timeout + retry(backoff) + breaker + fallback + idempotency**.

Boundaries by **business capability + data ownership**; each service owns its data.

Distributed transaction → **saga**, not 2PC; outbox for reliable publish.

Saga: **choreography** (events, decoupled) vs **orchestration** (a coordinator).

SQL / DB

Window functions (ROW_NUMBER/RANK) for per-group ranking.

Index the **WHERE / JOIN** columns; correlated subqueries are slow.

Scale order: **index → tune → replica → cache → partition → shard**.

CAP: under a partition you pick **consistency or availability**.

SYSTEM DESIGN

Sequence: **requirements → capacity → API → data model → components → bottlenecks → trade-offs**.

State assumptions and rough QPS **first**.

Make **checkout / payment idempotent**.

JAVA 17 / 21

Virtual threads for **IO-bound** concurrency; **don't pool** them; not for CPU-bound.

They **pin** the carrier inside `synchronized` or native calls — prefer locks.

Records are **shallow-immutable** data carriers.

Sealed types for **controlled hierarchies** + exhaustive switch.

BEHAVIORAL

Speak in the **first person ("I")**, not "we".

Lead with a number; have the rejected alternative ready.

Keep metrics **bounded and honest**.

30-DAY DAY-BY-DAY PLAN

Track 1 — Near-Term Interview Execution

Use this when interviews are near. Built for daily execution: 3–5 questions, two spoken answers, and one real-project connection — every day. Each day gives you what to **say** and the **trap** that separates a senior answer from a mid-level one.

· Goal & Rules

- ☐ Clear near-term coding and senior Java technical screens.
- ☐ **Coding method:** brute force → optimized → edge cases → complexity → dry run, out loud.
- ☐ **Theory method:** direct answer → mechanism → trade-offs → project example.
- ☐ **Output rule:** speak two answers out loud daily; get a real (human) mock in by the end of week 1.
- ☐ Do not skip project explanation — for lead roles it often decides the offer.

> **CONSCIOUS DSA CEILING**
>
> This plan targets the common contract bar — strings, arrays, HashMap, trees, searching, heap. If a target runs a harder algorithmic bar, add DP, graphs and intervals **deliberately** (see Track 4). Otherwise the time is better spent on Spring, Kafka and project depth.

Daily time split

AVAILABLE	HOW TO USE IT
60 min	35 min coding + 15 min Q&A spoken + 10 min project / weak-area log
90 min	50 min coding + 25 min Q&A and spoken answers + 15 min tracker
2+ hr	75 min coding / deep topic + 35 min Q&A and project + 20 min spoken answers + 10 min tracker

OPTIONAL Video or audio while commuting or on the treadmill — extra, never required.

· Week 1 — Coding Core

1 DAY

Strings / HashMap

Practice: first non-repeating char / duplicate characters / character frequency / anagram / string rotation

Say it (×2): Pick HashMap vs LinkedHashMap by whether order matters; state brute force first, then optimize.

> TRAP Override **equals() and hashCode() together** or HashMap lookups silently break; HashMap loses insertion order — use LinkedHashMap if order matters.

OPTIONAL Java string questions / immutability & string pool / StringBuilder vs StringBuffer

Done: ☐ 3–5 questions ☐ spoke 2 answers ☐ linked to a project ☐ logged weak area

2 DAY — Arrays

Practice: second highest / move zeroes / two sum / remove duplicates / merge two sorted arrays

Say it (×2): Brute force first, then optimize with a HashMap or two pointers.

> TRAP Confirm whether the array is **sorted** before reaching for two pointers; watch **integer overflow** in sum-based logic.

OPTIONAL two-pointer technique / HashMap optimization / complexity basics

Done: ☐ 3–5 questions ☐ spoke 2 answers ☐ linked to a project ☐ logged weak area

3 DAY — Sliding Window / Subarray

Practice: longest substring without repeat / Kadane / subarray with given sum / max sum subarray of size K / longest consecutive sequence

Say it (×2): Name the pattern before coding: fixed window, variable window, prefix sum, or Kadane.

> TRAP Decide **fixed vs variable window** up front — the bug is almost always in how you shrink the window.

OPTIONAL sliding window / prefix sum / Kadane

Done: ☐ 3–5 questions ☐ spoke 2 answers ☐ linked to a project ☐ logged weak area

4 DAY — Linked List

Practice: find middle / reverse / detect cycle / merge two sorted lists / remove Nth from end

Say it (×2): Slow/fast pointer avoids needing length; the dummy node kills edge-case bugs.

> TRAP Use a **dummy node** for any op that can change the head; null-check before `.next`.

OPTIONAL slow/fast pointers / dummy node / Floyd cycle

Done: ☐ 3–5 questions ☐ spoke 2 answers ☐ linked to a project ☐ logged weak area

5 DAY — LRU / Cache / HashMap Internals

Practice: LRU with LinkedHashMap / manual LRU (HashMap + DLL) / HashMap internals / equals/hashCode / cache with expiry

Say it (×2): Cache read-heavy data, never unsafe payment state.

> TRAP **LinkedHashMap(accessOrder=true)** gives LRU almost for free; a HashMap bucket treeifies at 8 entries (Java 8+).

DEEPER HashMap vs ConcurrentHashMap → Master Cheat Sheet

OPTIONAL LRU implementation / eviction strategies / hash collisions

Done: ☐ 3–5 questions ☐ spoke 2 answers ☐ linked to a project ☐ logged weak area

6 DAY — Threading

Practice: odd/even with two threads / producer-consumer (BlockingQueue) / deadlock + prevention / ExecutorService

Say it (×2): In production, prefer BlockingQueue and executors over manual wait/notify.

> TRAP **volatile = visibility, not atomicity** — use Atomic* or locks for compound actions.

OPTIONAL wait/notify / thread pools / BlockingQueue

Done: ☐ 3–5 questions ☐ spoke 2 answers ☐ linked to a project ☐ logged weak area

7 DAY — Week 1 Mock MOCK

Practice: one string / one array / one linked list / one threading / explain complexity

Say it (×2): Speak while coding: approach, edge cases, complexity, dry run. **Do this with a human if at all possible** (Pramp / interviewing.io / a peer), and pull one random topic from a past day.

OPTIONAL coding mock / speaking while coding

Done: ☐ 3–5 questions ☐ spoke 2 answers ☐ linked to a project ☐ logged weak area

· Week 2 — Streams, Search, Trees, Modern Java

8 DAY

Modern Streams — Employee Dataset JAVA 21

Practice: sort by salary / second-highest salary / group by department / max salary per dept / List to Map

Say it (×2): Streams help readability up to a point — don't force them on complex logic.

> TRAP This is modern Java, not just 8 — know `toList()`, `mapMulti`, `teeing`; streams are not automatically faster than a loop.

OPTIONAL groupingBy / map vs flatMap / Comparator.comparing

Done: ☐ 3–5 questions ☐ spoke 2 answers ☐ linked to a project ☐ logged weak area

9 DAY

More Streams / Collections

Practice: word frequency / duplicate elements / sort map by value / joining / partitioning

Say it (×2): Know groupingBy, partitioningBy, the toMap merge function, and Optional.

> TRAP **toMap throws on duplicate keys** without a merge function; fail-fast iterators throw ConcurrentModificationException.

OPTIONAL Collectors / Optional / when streams hurt readability

Done: ☐ 3–5 questions ☐ spoke 2 answers ☐ linked to a project ☐ logged weak area

10 DAY

Searching

Practice: binary search iterative/recursive / rotated sorted array / first/last occurrence / peak element

Say it (×2): Use overflow-safe mid = low + (high − low) / 2.

> TRAP Binary search is also a pattern on the **answer space** (min/max feasible value), not just on arrays.

OPTIONAL binary search pattern / edge cases

Done: ☐ 3–5 questions ☐ spoke 2 answers ☐ linked to a project ☐ logged weak area

11 DAY

Trees / BST

Practice: inorder/preorder/postorder / level-order / validate BST / height / LCA

Say it (×2): BST inorder gives sorted order; validate with a min/max range, not just parent comparison.

> TRAP Comparing only parent vs immediate child **passes invalid trees** — carry a (min, max) range down the recursion.

OPTIONAL BFS vs DFS / recursion / min/max validation

Done: ☐ 3–5 questions ☐ spoke 2 answers ☐ linked to a project ☐ logged weak area

12 DAY

Heap / PriorityQueue

Practice: kth largest / kth smallest / top-K frequent / merge K lists / median from stream (concept)

Say it (×2): For top-K, use a heap of size K: O(n log k).

> TRAP k-largest → **min-heap** of size k (not max-heap); k-smallest → max-heap. Easy to get backwards under pressure.

OPTIONAL PriorityQueue / top-K pattern / min vs max heap

Done: ☐ 3–5 questions ☐ spoke 2 answers ☐ linked to a project ☐ logged weak area

13 DAY

Java 17 / 21

Practice: records / sealed classes / pattern matching / switch expressions / virtual threads vs platform threads / limitations of virtual threads

Say it (×2): Be honest — many enterprises still run 8/11/17. Virtual vs platform: cheap, JVM-scheduled, mounted on a small pool of carrier threads.

> TRAP **Virtual threads help IO-bound** concurrency, not CPU-bound; **don't pool** them. They **pin** the carrier inside `synchronized` or native calls — use locks instead. Records are shallow-immutable.

DEEPER Concurrency & JVM depth → Track 4

OPTIONAL what changed after Java 8 / virtual threads

Done: ☐ 3–5 questions ☐ spoke 2 answers ☐ linked to a project ☐ logged weak area

---

14 DAY

Week 2 Mock MOCK

Practice: stream coding / binary search / tree / Java 17/21 explanation / one 25-min timed problem

Say it (×2): Solve, speak, test, stop rambling — the pressure is the point. Pull one random older topic too (spaced recall).

OPTIONAL senior coding mock

Done: ☐ 3–5 questions ☐ spoke 2 answers ☐ linked to a project ☐ logged weak area

---

· Week 3 — Spring, JPA, Kafka

15 DAY

Spring Core

Practice: IoC/DI / constructor injection / bean lifecycle / scopes / stereotypes / @Bean vs @Component

Say it (×2): Constructor injection for required dependencies — testable and immutable.

> TRAP **@Bean** = explicit / third-party config; **@Component** = classpath-scanned. Field injection hides circular deps and breaks tests.

DEEPER SOLID / DIP → Track 2 Wk 1

OPTIONAL Spring core interview questions

Done: ☐ 3–5 questions ☐ spoke 2 answers ☐ linked to a project ☐ logged weak area

---

16 DAY

Spring Boot

Practice: auto-configuration / starters / profiles / external config / actuator / ControllerAdvice

Say it (×2): Boot = convention + auto-config + starters + embedded server + production tooling.

> TRAP Auto-config is **conditional** (@ConditionalOnClass / OnMissingBean...). Externalize config via profiles/env, never hardcode.

DEEPER 12-factor config → Track 2 Wk 2

OPTIONAL auto-configuration / exception handling

Done: ☐ 3–5 questions ☐ spoke 2 answers ☐ linked to a project ☐ logged weak area

---

17 DAY

Transactions / JPA

Practice: @Transactional / propagation / isolation / self-invocation / N+1 / lazy/eager / locking

Say it (×2): Transactions are proxy-based — that is why self-invocation and private methods skip the proxy.

> TRAP Default propagation is **REQUIRED**; **self-invocation bypasses @Transactional**; fix N+1 with fetch join or @EntityGraph.

OPTIONAL transaction and JPA questions

Done: ☐ 3–5 questions ☐ spoke 2 answers ☐ linked to a project ☐ logged weak area

---

18 DAY

REST / Security

Practice: status codes / controller-service-repository / JWT / OAuth2 vs OIDC / filter chain / @Valid

Say it (×2): JWT validates claims/tokens; OIDC adds identity on top of OAuth2.

> TRAP Stateless auth = **no server session**; validate signature + expiry + audience on every request.

DEEPER Deeper auth (mTLS/PKCE) → Track 4 (Security)

OPTIONAL Spring Security JWT / REST design

Done: ☐ 3–5 questions ☐ spoke 2 answers ☐ linked to a project ☐ logged weak area

---

19 DAY

Kafka Basics

Practice: producer/broker/topic/partition / consumer group / offset / commit / lag / partition key / how Kafka achieves high throughput

Say it (×2): Partitioning gives parallelism, and ordering within a partition. Throughput comes from sequential disk IO, zero-copy, batching and the OS page cache.

> TRAP **Ordering holds only within a partition**; the key picks the partition; more partitions = more parallelism but rebalancing cost.

OPTIONAL Kafka basics / consumer groups / how Kafka is fast

Done: ☐ 3–5 questions ☐ spoke 2 answers ☐ linked to a project ☐ logged weak area

---

20 DAY

Kafka Senior

Practice: delivery semantics / duplicates / idempotent consumer / poison message / DLQ / rebalancing / broker failure (ISR + leader election) / Kafka vs RabbitMQ / safe event replay

Say it (×2): Real systems = at-least-once + idempotent processing. Broker dies → an in-sync replica is elected leader; with acks=all there is no data loss.

> TRAP Exactly-once is real **inside** Kafka (idempotent producer + transactions), but **end-to-end across external systems it is effectively at-least-once + dedup**. Say that distinction explicitly — it is the senior signal here.

DEEPER Resilience & saga/outbox → Day 23 / Track 2 Wk 6

OPTIONAL exactly-once / retries and DLQ / Kafka vs RabbitMQ

Done: ☐ 3–5 questions ☐ spoke 2 answers ☐ linked to a project ☐ logged weak area

---

21 DAY

Week 3 Mock MOCK

Practice: Spring / JPA / Kafka / WSI project example / one coding problem

Say it (×2): Every framework answer carries one real project anchor. Pull one random older topic (spaced recall).

OPTIONAL Spring/Kafka mock

Done: ☐ 3–5 questions ☐ spoke 2 answers ☐ linked to a project ☐ logged weak area

---

· Week 4 — Microservices, SQL, System Design, Projects

22 DAY

Microservices I

Practice: service decomposition / REST vs async / REST vs gRPC / API gateway / service discovery / config / load balancing

Say it (×2): Decide service boundaries by business capability and data ownership.

> TRAP **Each service owns its data** — no shared database. REST for sync, messaging for decoupled/async, gRPC for low-latency internal calls.

OPTIONAL microservices basics / API gateway / REST vs gRPC

Done: ☐ 3–5 questions ☐ spoke 2 answers ☐ linked to a project ☐ logged weak area

23 DAY

Microservices II / Resilience

Practice: downstream failure / circuit breaker / retry / timeout / bulkhead / saga choreography vs orchestration / outbox / eventual consistency / drawbacks of microservices / debug a slow microservice

Say it (×2): Downstream failure = timeout + retry with backoff + circuit breaker + fallback, and idempotency so the retries are safe.

> TRAP **Retries without idempotency cause duplicates**; use a **saga** for distributed transactions, not 2PC. **Choreography** (events, decoupled) vs **orchestration** (a central coordinator, easier to reason about) — know the trade-off.

DEEPER Runbooks & observability → Track 2 Wk 6

OPTIONAL Resilience4j / saga choreography vs orchestration / outbox

Done: ☐ 3–5 questions ☐ spoke 2 answers ☐ linked to a project ☐ logged weak area

24 DAY

SQL

Practice: second/Nth highest / max salary by dept / joins / group by / window functions / indexing / tuning

Say it (×2): Write queries by hand. Reach for window functions (ROW_NUMBER/RANK) for per-group ranking.

> TRAP **Index the WHERE / JOIN columns**; correlated subqueries are slow; HAVING filters aggregates, WHERE filters rows.

OPTIONAL SQL interview questions / window functions

Done: ☐ 3–5 questions ☐ spoke 2 answers ☐ linked to a project ☐ logged weak area

25 DAY

DB Scaling / NoSQL

Practice: RDBMS vs NoSQL / read replicas / sharding / partitioning / Cassandra vs PostgreSQL / CAP

Say it (×2): Relational scaling order: indexing → query tuning → read replicas → caching → partitioning → sharding.

> TRAP **Shard last**, not first; under a partition CAP forces a choice of consistency or availability; Cassandra is query-first (model to the read).

DEEPER Cassandra internals → Track 4 (Deep NoSQL)

OPTIONAL sharding / replication / CAP

Done: ☐ 3–5 questions ☐ spoke 2 answers ☐ linked to a project ☐ logged weak area

26 DAY

System Design — eCommerce

Practice: shopping cart / checkout / gift-card inquiry / fraud/risk / idempotent payment API

Say it (×2): Closest to your WSI experience — own it. Sequence: requirements → capacity → API → data model → components → bottlenecks → trade-offs.

> TRAP Make **checkout and payment idempotent**; separate the read path (catalog/cart) from the write path (order/payment).

OPTIONAL design checkout / payment system

Done: ☐ 3–5 questions ☐ spoke 2 answers ☐ linked to a project ☐ logged weak area

---

27 DAY

System Design — General

Practice: news feed / URL shortener / notification system / rate limiter / distributed cache

Say it (×2): Same sequence every time; state assumptions and rough QPS first, then find the bottleneck.

> TRAP Don't jump to the data model — **requirements and scale assumptions first**, or you design the wrong system.

OPTIONAL design Twitter / URL shortener

Done: ☐ 3–5 questions ☐ spoke 2 answers ☐ linked to a project ☐ logged weak area

---

28 DAY

Project Deep Dive — WSI

Practice: cart/checkout / Accertify / Kafka / Tech Anchor / production issue / design decision

Say it (×2): Context → problem → your role → solution → impact (bounded number). Two minutes, no rambling.

> TRAP Own it in the **first person ("I")**; lead with a number; have the rejected alternative ready for the follow-up.

DEEPER STAR-plus template → Master §9

OPTIONAL senior project walkthrough

Done: ☐ 3–5 questions ☐ spoke 2 answers ☐ linked to a project ☐ logged weak area

---

29 DAY

Project Deep Dive — CARB / Payments

Practice: Struts → Spring Boot / monolith → microservices / AWS migration / Cognito/OAuth/JWT / ISO8583

Say it (×2): Use CARB and payments as your older-depth stories.

> TRAP Keep the narrative to current and recent roles; in payments, idempotency rides on the **business / transaction key**.

OPTIONAL migration and payment questions

Done: ☐ 3–5 questions ☐ spoke 2 answers ☐ linked to a project ☐ logged weak area

---

30 DAY

Full Mock MOCK

Practice: coding / Spring / Kafka / SQL / system design / project walkthrough / behavioral

Say it (×2): Clear opening line, structured body, real example, no rambling. Score yourself on the Master's Mock Scorecard. Pull two random older topics (spaced recall).

OPTIONAL full senior Java mock

Done: ☐ 3–5 questions ☐ spoke 2 answers ☐ linked to a project ☐ logged weak area

---

· Expanded Coding Practice Bank

Build the muscle to write these from scratch — plain loops, primitive arrays, no built-in collections. Each problem appears once, in its best home; work Tier 1 top-to-bottom first, then the topical sections (which reuse the same primitives). Do them without looking; if you stall, reach for a toolkit technique below, not a solution.

> **PLAIN-LOOP TOOLKIT (REACH FOR THESE, NOT A HASHMAP)**
>
> **Two pointers** from both ends — reverse, palindrome, in-place swap. · **Frequency array** — `int[256]` (ASCII) or `int[26]` (lowercase) replaces a HashMap for counting. · **One-pass running state** — track min / max / second-max / sum / count as you scan once. · **Write-index** — a second pointer marks where the next kept element goes (compaction in place). · **Prefix accumulation** — running sum or product. · **Fast / slow pointers** — middle of a list, cycle detection. · **Math tricks** — `n(n+1)/2` sum and **XOR** for missing / duplicate numbers. · **Brute force first** (nested loops), then optimize.

Tier 1 — Plain-Loop Fundamentals (no built-in data structures)

1. reverse (string / array / int)
2. palindrome (string / number)
3. character frequency (int[256])
4. first non-repeating char (two passes)
5. anagram check (int[26])
6. find duplicate characters
7. count vowels and consonants
8. remove duplicate characters (seen flags)
9. reverse words in a sentence (no split)
10. count words in a string
11. find max and min in one pass
12. find second largest in one pass
13. left-rotate an array by K (reversal)
14. remove duplicates from sorted array (in place)
15. move zeroes to end (in place)
16. sum and average without overflow
17. count occurrences of a value
18. naive substring search (strstr)
19. check two arrays are equal
20. missing number (sum or XOR)
21. single non-repeating number (XOR)
22. count digits and sum of digits
23. swap two numbers without temp
24. GCD (Euclid)
25. Fibonacci (two variables)
26. factorial (iterative)
27. prime check (trial division)
28. multiplication table

Strings

1. string rotation check
2. first repeated char
3. longest substring without repeat
4. longest substring with K distinct
5. run-length compression
6. run-length decode
7. longest common prefix
8. valid palindrome (ignore non-alphanumeric)
9. group anagrams
10. isomorphic strings
11. most repeated word
12. remove given characters
13. replace spaces with %20
14. capitalize each word
15. count substring occurrences
16. one-edit distance
17. longest palindromic substring (basic)
18. string to int (atoi)
19. int to string (itoa)
20. swap case

Numbers / Math

1. digital root
2. LCM
3. primes up to N (sieve)
4. Armstrong number
5. perfect number
6. count set bits
7. power(x, n) fast exponentiation
8. is power of two
9. decimal to binary
10. binary to decimal
11. reverse bits
12. add two number-strings
13. multiply without *
14. divide without /
15. sum of first N naturals
16. nth prime
17. max of two without if
18. trailing zeroes in factorial

Arrays

1. two sum
2. three sum (basic)
3. missing and repeating
4. merge two sorted arrays
5. sort 0/1/2 (Dutch flag)
6. leaders in array
7. majority element (Boyer-Moore)
8. max subarray (Kadane)
9. max product subarray
10. subarray with sum K
11. product except self
12. longest consecutive sequence
13. minimum absolute difference
14. equilibrium index
15. best time to buy/sell stock
16. trapping rain water
17. merge intervals
18. pair with given difference
19. rearrange positive and negative
20. next permutation
21. find the duplicate (Floyd)
22. count subarrays with sum K
23. min jumps to reach end

Matrix / 2D

1. transpose
2. rotate 90 degrees
3. spiral traversal
4. print boundary
5. search in sorted matrix
6. set matrix zeroes
7. diagonal sums
8. row with max ones
9. count islands (basic)
10. flood fill (basic)
11. matrix multiplication
12. symmetric matrix check
13. snake traversal
14. max sum submatrix (basic)
15. number of distinct paths

Searching & Sorting

1. linear search
2. binary search (iterative / recursive)
3. first and last occurrence
4. search in rotated sorted array
5. find peak element
6. integer square root (binary)
7. ceiling and floor in sorted
8. bubble sort
9. selection sort
10. insertion sort
11. merge sort
12. quick sort
13. counting sort
14. find rotation count (min in rotated)
15. allocate minimum pages (binary on answer)
16. min-max placement (binary on answer)
17. median of two sorted (basic)

Recursion / Backtracking

First re-solve factorial, Fibonacci, power and GCD from Tier 1 recursively — then:

1. tower of Hanoi
2. subsets / subsequences (power set)
3. permutations
4. combinations (n choose k)
5. generate parentheses
6. N-Queens (basic)
7. rat in a maze
8. word search in grid (basic)

Stack / Queue

1. implement stack with array
2. implement queue with array
3. min stack
4. valid parentheses / balanced brackets (multi-type)
5. next greater element
6. stack using two queues
7. queue using two stacks
8. evaluate postfix
9. infix to postfix (basic)
10. circular queue
11. sliding window maximum (deque)
12. largest rectangle in histogram
13. stock span

Linked List

1. reverse a linked list
2. find middle
3. detect cycle
4. find cycle start
5. merge two sorted
6. remove Nth from end
7. palindrome list
8. intersection node
9. delete node (given pointer)
10. reverse in K groups
11. add two numbers (lists)
12. remove duplicates (sorted)
13. remove duplicates (unsorted)
14. rotate by K
15. odd-even segregation
16. flatten (basic)

Trees / BST

1. inorder / preorder / postorder
2. level order (BFS)
3. height
4. diameter
5. count nodes and leaves
6. validate BST
7. lowest common ancestor
8. BST insert and search
9. BST delete (basic)
10. invert / mirror
11. left view and right view
12. zigzag traversal
13. root-to-leaf path sum
14. kth smallest in BST
15. balanced tree check
16. serialize / deserialize (basic)

Heap / PriorityQueue

1. kth largest / smallest
2. top-K frequent
3. merge K sorted lists

4. median from data stream
5. sort a K-sorted array
6. K closest points
7. connect ropes (min cost)
8. reorganize string (basic)
9. task scheduler (basic)

Concurrency

1. odd/even with two threads
2. print in sequence (3 threads)
3. producer-consumer (BlockingQueue)
4. producer-consumer (wait/notify)
5. create and prevent deadlock
6. thread-safe singleton (double-checked)
7. ExecutorService example
8. CompletableFuture composition
9. ConcurrentHashMap example
10. simple rate limiter
11. implement a bounded blocking queue
12. atomic counter vs synchronized

· Interview-Week Technical Q&A Drill

Speak each answer out loud with one project anchor. This is a drill list, not reading material.

Spring · Kafka · Microservices · SQL

1. Spring DI and constructor injection
2. Bean lifecycle and scopes
3. @Bean vs @Component
4. Spring Boot auto-configuration
5. @Transactional propagation / isolation
6. self-invocation transaction issue
7. N+1 and fetch strategies
8. JWT / OAuth / OIDC
9. Spring Security filter chain
10. REST status codes and error response
11. Kafka partitions and ordering
12. consumer groups and lag
13. offset commit strategies
14. delivery semantics
15. idempotent consumer
16. DLQ and poison messages
17. retry-storm prevention
18. how Kafka achieves high throughput
19. Kafka broker failure (ISR / leader election)
20. Kafka vs RabbitMQ
21. safe event replay
22. API gateway / service discovery
23. circuit breaker and fallback
24. saga and outbox
25. saga choreography vs orchestration
26. eventual consistency
27. REST vs gRPC
28. debug a slow microservice
29. idempotency-key design
30. SQL joins
31. window functions
32. indexing and query tuning
33. RDBMS vs NoSQL
34. scaling a relational DB
35. shopping-cart design
36. checkout design
37. rate-limiter design
38. notification-system design
39. SOLID / OCP
40. 12-factor app
41. observability / correlation ID
42. CI/CD rollback
43. a production-incident story

Collections & Internals

1. HashMap internals
2. HashMap vs ConcurrentHashMap
3. HashSet vs TreeSet
4. ArrayList vs LinkedList
5. LinkedHashMap vs TreeMap
6. CopyOnWriteArrayList
7. WeakHashMap use cases
8. fail-fast vs fail-safe iterators
9. why HashMap is not thread-safe
10. how ConcurrentHashMap is thread-safe
11. Comparable vs Comparator

Concurrency Q&A

1. CompletableFuture vs Future
2. synchronized vs ReentrantLock

3. AtomicInteger vs synchronized
4. CountDownLatch vs CyclicBarrier
5. what causes deadlocks
6. thread starvation vs livelock
7. CAS (compare-and-swap)
8. ExecutorService vs virtual threads
9. ThreadPoolExecutor tuning
10. volatile vs atomic

Modern Java (21 / 25)

1. virtual threads vs platform threads
2. when NOT to use virtual threads
3. structured concurrency
4. scoped values
5. records (what they solve)
6. pattern matching for switch
7. sealed types
8. sequenced collections
9. stream gatherers
10. string templates (preview)
11. foreign function & memory API

Core Java & OOP Fundamentals

1. pass-by-value (can you swap two ints?)
2. Integer caching (-128 to 127)
3. == vs equals
4. final vs finally vs finalize
5. immutability + writing an immutable class
6. why String is immutable / a good key
7. static method hiding vs overriding
8. overloading vs overriding
9. checked vs unchecked exceptions
10. Errors vs Exceptions (name 3 each)
11. constructor chaining (this / super)
12. marker interface (Serializable, Cloneable)
13. transient + Serialization
14. type erasure + primitives as generics
15. why no multiple inheritance (diamond)
16. BigDecimal for money (not double)
17. autoboxing / unboxing pitfalls
18. JDK vs JRE vs JVM
19. JVM memory: heap / stack / method area
20. String vs StringBuilder vs StringBuffer
21. Runnable vs Callable
22. tail recursion

· Track 1 Coverage Check

- ☐ Solved 20+ coding questions without looking at solutions.
- ☐ Can implement LRU or explain two approaches.
- ☐ Can handle the streams employee questions.
- ☐ Can answer Spring transaction and JPA N+1 questions.
- ☐ Can explain Kafka delivery semantics and duplicate handling.
- ☐ Can write SQL window-function queries.
- ☐ Can design cart/checkout/rate limiter at high level.
- ☐ Can explain WSI and CARB in under 2 minutes each.

> **COVERAGE IS NECESSARY, NOT SUFFICIENT**
>
> The real gauge is your Mock Scorecard trend in the Master. You are ready when you hit 4–5 consistently on questions you have not pre-seen — not when this list is full.

12-WEEK PRODUCTION-READINESS & LEADERSHIP PLAN

Track 2 — Senior / Tech Lead Mastery

This turns you from a Java developer into a production-ready tech-lead candidate. Every answer must carry a project example from WSI, CARB or payments. Run it on Saturdays plus 20 minutes on weekdays.

· Goal & Operating Model

- ☐ Explain maintainability, production readiness and design quality clearly.
- ☐ Build 5 short answers per week and **speak each twice**.
- ☐ Tie every concept to a real project (WSI / CARB / payments).

> **INTERVIEW-WEEK FLOOR**
>
> When Track 1 takes over for an interview, keep a single **20-minute Track 2 touch** for the week — one senior concept + one project example — so depth never drops to zero. Non-negotiable; everything else can pause.

· The 12-Week Senior / Lead Build

Week 1 — SOLID Principles

Explain SOLID with real project examples, not textbook definitions.

STUDY

- ☐ SRP
- ☐ OCP
- ☐ LSP
- ☐ ISP
- ☐ DIP

FRAME IT WITH

- ☐ Controller-service-repository separation
- ☐ Payment/fraud strategy extension without touching core flow
- ☐ Subclasses don’t break parent behavior
- ☐ Small interfaces per client
- ☐ Spring DI on interfaces, not concretes

This week: Write 5 short answers and speak each twice. Signal: name where over-applying a principle would hurt.

Week 2 — 12-Factor App

Explain cloud-native Spring Boot service design.

STUDY

- ☐ Codebase, dependencies, config
- ☐ Backing services, build/release/run
- ☐ Processes, port binding, concurrency
- ☐ Disposability, dev/prod parity
- ☐ Logs, admin processes

FRAME IT WITH

- ☐ Profiles + env vars + secrets per environment
- ☐ Dockerized Spring Boot service
- ☐ Stateless REST APIs
- ☐ Actuator health checks
- ☐ CI/CD separates build / release / run

This week: Write 5 short answers and speak each twice.

Week 3 — Design Patterns

Explain patterns through real project examples.

STUDY

- [ ] Factory
- [ ] Strategy
- [ ] Builder
- [ ] Singleton
- [ ] Observer
- [ ] Decorator
- [ ] Template
- [ ] Adapter

FRAME IT WITH

- [ ] Strategy: payment/fraud rules
- [ ] Factory: client/processor by type
- [ ] Builder: Accertify/device payload
- [ ] Singleton: Spring beans
- [ ] Observer: Kafka event flow
- [ ] Adapter: external-client wrapper

This week: Write 5 short answers and speak each twice. Signal: when a simpler alternative beats the pattern.

Week 4 — API Design & Clean Architecture

Design clean, maintainable REST services.

STUDY

- [ ] DTOs vs entities
- [ ] Validation, error model
- [ ] Versioning, pagination
- [ ] Idempotency keys, correlation IDs
- [ ] API contracts, backward compatibility

FRAME IT WITH

- [ ] Cart/checkout API design
- [ ] Global exception model
- [ ] Idempotent payment endpoint
- [ ] DTO mapping hides entity internals
- [ ] Correlation ID across logs

This week: Write 5 short answers and speak each twice.

Week 5 — DDD-lite / Domain Modeling

Model the domain, not just tables.

STUDY

- [ ] Bounded context
- [ ] Aggregate, value object
- [ ] Domain service, repository
- [ ] Anti-corruption layer

FRAME IT WITH

- [ ] Cart, checkout, payment, fraud/risk, order boundaries
- [ ] Aggregates around invariants
- [ ] ACL around legacy/3rd-party

This week: Write 5 short answers and speak each twice. Signal: where a wrong boundary caused coupling.

Week 6 — Observability & Production Readiness

Talk like someone who supports production.

STUDY

- [ ] Structured logs, correlation ID
- [ ] Metrics, traces
- [ ] Health checks, alerts, dashboards
- [ ] Runbooks, incident triage

FRAME IT WITH

- [ ] Checkout latency, Kafka lag
- [ ] Accertify downstream errors
- [ ] CloudWatch / ELK / Splunk
- [ ] Runbook for downstream failure

This week: Write 5 short answers and speak each twice.

Week 7 — Testing Strategy

Explain how you ensure quality as a lead.

STUDY

- [ ] Unit, Mockito
- [ ] Integration, TestContainers
- [ ] Contract, API, performance
- [ ] Coverage and review gates

FRAME IT WITH

- [ ] JUnit/Mockito service tests
- [ ] Kafka integration test
- [ ] Sonar/Jacoco gates
- [ ] Code-review checklist

This week: Write 5 short answers and speak each twice.

Week 8 — CI/CD & DevOps

Explain deployment flow and rollback.

STUDY

- [ ] Jenkins, Maven/Gradle
- [ ] Docker, K8s, probes
- [ ] Secrets
- [ ] Blue-green / canary, rollback

FRAME IT WITH

- [ ] Pipeline stages
- [ ] Readiness/liveness probes
- [ ] Release validation
- [ ] Rollback strategy

This week: Write 5 short answers and speak each twice.

Week 9 — Security

Explain secure backend design.

STUDY

- [ ] AuthN / AuthZ
- [ ] JWT, OAuth2, OIDC, RBAC
- [ ] OWASP, SQL-injection
- [ ] Secrets management

FRAME IT WITH

- [ ] Cognito/JWT in CARB
- [ ] JWT filter chain
- [ ] RBAC by role/claim
- [ ] Never log secrets/tokens

This week: Write 5 short answers and speak each twice.

Week 10 — Performance & Scalability

Find bottlenecks and scale safely.

STUDY

- [ ] Profiling, indexes
- [ ] Cache-aside, connection/thread pools
- [ ] Async, backpressure
- [ ] Kafka scaling, load testing

FRAME IT WITH

- [ ] Checkout latency triage
- [ ] HikariCP pool
- [ ] Kafka partitions/consumers
- [ ] JMeter baseline

This week: Write 5 short answers and speak each twice.

Week 11 — Leadership & Delivery

Prepare lead-level stories and behaviors.

STUDY

- [ ] Mentoring, reviews, estimation
- [ ] Conflict, incidents
- [ ] Tech debt, stakeholder comms
- [ ] Decision records

FRAME IT WITH

- [ ] Tech Anchor at WSI
- [ ] Offshore coordination
- [ ] Driving a decision
- [ ] Production incident ownership

This week: Write 5 lead-level stories and speak each twice.

Week 12 — Architecture Review Board Mode

Drive design decisions across teams.

STUDY

- [ ] Decision records (ADRs)
- [ ] Trade-off docs, standards
- [ ] Governance
- [ ] Migration roadmap

FRAME IT WITH

- [ ] How you drove a cross-team decision
- [ ] A migration you sequenced
- [ ] Standards you set or enforced

This week: Write 3 decision-record style answers and speak each.

· Project Mapping Table

CONCEPT	PROJECT EXAMPLE
SRP	Controller-service-repository separation in checkout APIs.
OCP	Payment/fraud rules via Strategy so new rules don't change orchestration.
Builder	Accertify / device payload creation.
Adapter	Wrapper around an external fraud/risk client.
Observer	Kafka event flow for risk / eGift / gift-card events.
12-factor config	Profiles / env vars / secrets per environment.
Idempotency	Payment/checkout/Kafka processing on a business key / event ID.
Observability	Correlation ID from API to downstream logs; Kafka lag dashboard.
Security	Cognito/OAuth/JWT in CARB; JWT filter chain in Spring Security.
Leadership	Tech Anchor: reviews, offshore coordination, decisions, delivery.

· Senior / Tech-Lead Question Bank

SOLID / Patterns

1. Explain OCP with a real example.
2. Where did you use Strategy?
3. Factory vs Strategy.
4. How is Builder useful for payloads?
5. Spring singleton beans vs the Singleton pattern.
6. Adapter for third-party integration.
7. How do you avoid over-engineering patterns?
8. How do you apply DIP in Spring?
9. Risk of violating SRP.
10. How do you review code for SOLID?

Architecture / API

1. How do you design a REST API?
2. DTO vs entity.
3. Error response format.
4. API versioning strategy.
5. Idempotency-key design.
6. Correlation-ID propagation.
7. Pagination and sorting.
8. Backward compatibility.
9. How do you split services?
10. How do you document API contracts?

Production

1. How do you debug production issues?
2. What logs do you add?
3. Metrics you track for checkout.
4. How do you monitor Kafka lag?
5. How do you design alerts?
6. Runbook for a downstream failure.
7. Incident communication.
8. How do you validate a release?
9. A good rollback plan.
10. How do you handle tech debt?

Testing / Quality

1. Unit vs integration tests.
2. Mockito best practices.
3. TestContainers usage.
4. Contract testing.
5. Kafka testing strategy.
6. API testing.
7. Performance testing.
8. Sonar/Jacoco gates.
9. Code-review checklist.
10. Mentoring juniors on testing.

Leadership

1. How do you lead offshore teams?
2. How do you handle disagreement?
3. How do you estimate work?
4. How do you split tasks?
5. How do you mentor?
6. How do you run code reviews?
7. How do you make design decisions?
8. How do you handle missed deadlines?
9. How do you communicate risk?
10. Why hire you as a lead?

· Answer Templates

TOPIC	TEMPLATE
SOLID	Definition → project example → benefit → trade-off → how you enforced it in reviews.
12-factor	Principle → Spring Boot implementation → production benefit → mistake to avoid.
Design pattern	Problem → pattern → implementation → why not a simpler alternative → project usage.
Observability	Logs + metrics + traces + correlation ID + alert + runbook + a project incident.
Leadership	Situation → decision → collaboration → measurable outcome → lesson learned.

· Production Debugging Scenarios

The format senior/lead interviews actually use. Don’t jump to a fix — walk the method out loud, then narrow to the likely layer.

> **DIAGNOSTIC LADDER**
>
> Reproduce / confirm the symptom → look at **metrics and dashboards** first → narrow the layer (app code, GC, threads, DB, downstream, network) → form a hypothesis → verify with **logs / traces / heap or thread dumps** → fix → add a guardrail (alert, test, runbook) so it cannot silently recur.

1. API latency jumped 200ms → 5s — how do you debug?
2. JVM memory grows continuously — find the leak
3. CPU pinned at 100% — what do you check?
4. Database slow right after a deploy
5. Thread-pool queue keeps growing
6. Kafka consumer lag climbing — how do you fix it?
7. Checkout p95 regressed overnight
8. A downstream dependency started timing out
9. Design a cache for 10M users
10. Design a rate limiter with Redis

· Track 2 Coverage Map

- ☐ Explain all SOLID principles with project examples.
- ☐ Explain the 12 factors in Spring Boot terms.
- ☐ Map Strategy / Factory / Builder / Adapter to a real project.
- ☐ Design a clean REST API: DTOs, validation, errors, versioning, idempotency.
- ☐ Explain logs, metrics, traces, correlation IDs and dashboards.
- ☐ Explain testing: unit, integration, Kafka/API and quality gates.
- ☐ Explain CI/CD, Docker, K8s, probes, rollback and release validation.
- ☐ Explain JWT / OAuth / OIDC / RBAC and secure API design.
- ☐ Explain domain boundaries and where a wrong one caused coupling.
- ☐ Tell 5 lead-level behavioral stories and drive one design decision.

> **READINESS**
>
> These boxes mean you have *covered* the topic. The gauge is whether you can deliver each in a mock — score them on the Master's Mock Scorecard.

3-6 MONTH LONG-TERM PLAN

Track 3 — Java + AI/RAG Differentiator

This moves you from regular Java Lead to Java + AI/RAG + architecture candidate. It is for stronger roles, better rates and long-term market value — not last-minute panic.

· Goal & Positioning

POSITIONING STATEMENT

“My core is Java / Spring Boot microservices. AI/RAG is an extension layer I am building on top of backend systems — ingest, chunk, embed, retrieve, augment and generate grounded answers. I frame it honestly as a growing differentiator, not as inflated AI production experience.”

- ☐ Build small **finished** proof projects, not big unfinished demos.
- ☐ Use AI/RAG to strengthen eCommerce and backend positioning.
- ☐ Publish GitHub / LinkedIn / Medium proof gradually.
- ☐ Do not oversell production AI you have not built in production.

> **THE BAR**
>
> One small, **finished**, public artifact with a clean README beats four half-written drafts. Finished and small > ambitious and abandoned. Ship one thing, then the next.

· The 6-Month Roadmap

Month 1 — AI/RAG Foundations

Build vocabulary and an honest understanding of GenAI for backend roles.

STUDY

- ☐ Tokens, context window, temperature
- ☐ Embeddings, vector search
- ☐ Semantic search, chunking
- ☐ Retrieval quality

OUTPUT TO FINISH

- ☐ Explain the RAG pipeline in 2 min: ingest → chunk → embed → retrieve → augment → generate
- ☐ Explain hallucination and grounding

Month 2 — Spring AI Practical Layer

Build one small, finished RAG project on Spring Boot.

STUDY

- ☐ ChatClient, prompt templates
- ☐ Embedding model, pgvector
- ☐ Vector-store abstraction
- ☐ Evaluation, guardrails, security

OUTPUT TO FINISH

- ☐ **Working mini app:** upload → chunk → embed → ask → **cited answer**
- ☐ A small evaluation set + a clean README

Month 3 — eCommerce AI Use Cases

Connect AI/RAG to your eCommerce profile.

STUDY

- ☐ Gift reminders, occasion intelligence
- ☐ Product classification
- ☐ Recommendations, search relevance

OUTPUT TO FINISH

- ☐ One small gift-reminder or product-classification demo
- ☐ Frame honestly: core is Java/Spring, AI is the edge

Month 4 — Advanced Event-Driven

Go deeper in Kafka and distributed processing.

STUDY

- ☐ Ordering, retries, DLQ
- ☐ Schema evolution, event versioning
- ☐ Outbox, idempotency, observability

OUTPUT TO FINISH

- ☐ **Finish a Kafka retry/DLQ demo**
- ☐ One post on event-driven trade-offs

Month 5 — Data Modeling / Distributed Systems

Strengthen architecture depth.

STUDY

- ☐ Cassandra, consistency models
- ☐ Caching, sharding
- ☐ Backpressure, distributed locks

OUTPUT TO FINISH

- ☐ Cassandra data-model examples
- ☐ A scalable checkout/order read path

Month 6 — Public Proof

Create evidence recruiters and clients can see.

STUDY

- ☐ GitHub, README, diagrams
- ☐ Posts, mock stories
- ☐ Resume alignment

OUTPUT TO FINISH

- ☐ Publish 2–3 finished artifacts
- ☐ Record 2-minute explanations

· AI/RAG Interview Question Bank

Drill as spoken answers, framed at your honest level.

1 What is RAG?
2 What are embeddings?
3 What is vector search?
4 Semantic vs keyword search.
5 What is chunking?
6 How do you decide chunk size?
7 What is top-k retrieval?
8 What is reranking?
9 What is hallucination?
10 How do you reduce hallucination?
11 What is grounding?
12 What is temperature?
13 What is a context window?
14 How do you secure GenAI apps?
15 How do you avoid sending PII?
16 How would you add AI to a Spring Boot app?
17 What is Spring AI ChatClient?
18 How does a vector-store abstraction work?
19 pgvector vs a dedicated vector DB (high-level).
20 How do you evaluate RAG quality?
21 How would you build document Q&A?
22 How would you build product recommendation?
23 How would you build a gift-reminder AI?
24 How do you monitor LLM usage?
25 How do you cache LLM responses?
26 How do you handle prompt injection?
27 How do you version prompts?
28 How do you test AI output?
29 How do you handle cost / latency?
30 How do you explain AI experience honestly?

· Portfolio Proof Checklist

- ☐ Spring AI RAG Q&A mini-project with a clean README.
- ☐ Kafka retry / DLQ / idempotent-consumer demo.
- ☐ eCommerce gift-reminder AI mini-project.
- ☐ Cassandra data-modeling notes or article.
- ☐ System-design diagram for checkout / risk evaluation.
- ☐ LinkedIn post on Kafka idempotency.
- ☐ Medium article on Cassandra-vs-RDBMS thinking.
- ☐ GitHub README polished with diagram and run steps.

· Project Ideas

1. Document Q&A with citations
2. Gift-reminder occasion classifier
3. Product classification from title/description
4. Cart-abandonment assistant
5. Customer-support RAG over FAQs
6. Fraud-explanation assistant (high-level)
7. Search-relevance re-ranker (high-level)
8. Kafka event summarizer
9. Architecture-decision assistant
10. Resume / JD matcher

· What Not To Claim

> **CREDIBILITY GUARDRAIL**
>
> This protects you from getting dismantled on the one area you can't fully back. Keep it strict.

- ☐ Do not claim **production GenAI ownership** if it was only a demo.
- ☐ Do not claim **model training** unless you actually trained / fine-tuned a model.
- ☐ Do not claim **vector-DB production expertise** until you build one and can explain the trade-offs.
- ☐ Say **"I built / explored / prototyped"** when that is the honest level.

· Track 3 Coverage Map

- ☐ Explain RAG clearly in 2 minutes.
- ☐ Explain embeddings and vector search.
- ☐ Describe Spring AI components at a high level.
- ☐ Built or can whiteboard a document-Q&A RAG system.
- ☐ Built or can whiteboard an eCommerce AI use case.
- ☐ Explain Kafka retries, DLQ, outbox and idempotency deeply.
- ☐ Explain Cassandra data-modeling basics.
- ☐ Have at least 2 finished GitHub demos or diagrams.
- ☐ Have a few public posts or solid drafts.
- ☐ Position AI honestly, without overselling.

CONDITIONAL ROADMAP FOR ADVANCED DEPTH

Track 4 — Future Reference / Advanced Gaps

A map of advanced areas to learn **later**, not a new burden. Track 4 is not for current interviews unless a job description clearly demands it.

· Learn-Only-If Labels

LABEL	MEANING
JD asks	Learn only when a job description or client clearly needs it.
Product-company bar	Learn for algorithm-heavy / FAANG-style rounds.
Cloud / platform role	Learn for DevOps / SRE / platform-heavy roles.
Frontend-heavy role	Learn for React / UI-heavy interviews.
Architect path	Learn for principal / architect roles.

· Advanced Missing Areas

AREA	HIGH-LEVEL TOPICS	WHEN TO LEARN
Advanced DSA	DP, graphs, intervals, trie, union-find, topological sort, Dijkstra, deeper backtracking, segment tree (high-level).	Product-company bar
Deep React / Frontend	Advanced hooks, state architecture, React Query, Redux Toolkit, SSR/ Next.js, accessibility, FE testing, performance profiling.	Frontend-heavy role
Deep AWS	VPC, subnets, security groups, IAM detail, ECS/EKS, Lambda architecture, API Gateway, CloudFront, Route53, multi-region DR.	Cloud / platform role
Kubernetes / SRE	Helm, ingress, HPA/VPA, PDB, service mesh, requests/limits, SLO/ SLA, error budgets, incident command.	Cloud / platform role
JVM Internals	GC algorithms, heap/thread dumps, JFR, memory leaks, class loading, JIT, JVM flags, tuning.	JD asks / architect
Advanced Distributed Systems	Consensus, leader election, quorum, vector clocks, distributed locks, backpressure, consistency models, CRDTs (high-level).	Architect path
Deep NoSQL / Search	Cassandra compaction/tombstones/repair, Elasticsearch/OpenSearch, Redis internals, Mongo aggregation.	JD asks
Security / Compliance	mTLS, cert rotation, deeper OAuth flows, PCI, OWASP ASVS, threat modeling, audit logging, secrets rotation.	Security-heavy role
Data Engineering	Spark, Flink, Kafka Streams, CDC/Debezium, lakehouse, batch vs stream, schema registry.	Data / platform role
Architecture Governance	ADRs, standards, review boards, migration roadmaps, platform strategy, cost governance.	Architect path

· Advanced Question Bank — Future Reference

Algorithms

1. DP memoization vs tabulation
2. Graph BFS vs DFS
3. Detect cycle in directed graph
4. Topological sort
5. Dijkstra (high-level)
6. Merge intervals
7. Trie use cases
8. Union-find use cases
9. Coin change
10. Longest common subsequence

Kubernetes

1. What happens when a Pod crashes
2. Deployment vs StatefulSet vs DaemonSet
3. Service discovery in K8s
4. Readiness vs liveness probes
5. Troubleshoot CrashLoopBackOff
6. What causes pod eviction
7. HPA vs VPA
8. Scale a Java app in K8s
9. Manage secrets securely
10. What happens during a rolling deployment
11. Ingress controller
12. Service mesh

AWS

1. EC2 vs ECS vs EKS
2. When to choose Lambda over ECS
3. SQS vs SNS
4. How S3 achieves durability
5. Secure applications in AWS
6. RDS vs DynamoDB
7. AWS Auto Scaling
8. Design a highly available app in AWS
9. ALB vs NLB
10. Troubleshoot high latency in AWS
11. VPC / subnets / security groups
12. IAM least privilege
13. Multi-region DR (RPO/RTO)

> **AZURE, NOT AWS — POSITION DELIBERATELY**
>
> Your production cloud depth is **Azure** (WSI was Azure-only), so a polished "senior AWS" question list is a positioning trap. Answer AWS questions by mapping to the Azure services you have actually run — and say so: ECS/EKS → AKS, Lambda → Functions, SQS/SNS → Service Bus / Event Grid, DynamoDB → Cosmos DB, RDS → Azure SQL — or frame AWS conceptually. Do not imply AWS production experience you do not have (your own Track 3 "What Not to Claim" rule).

JVM / Performance

1. G1 vs ZGC vs Shenandoah — when to choose each
2. What happens during a Full GC
3. Java Memory Model (JMM)
4. How ForkJoinPool works
5. False sharing and its performance impact
6. Profile a production JVM
7. Analyze a heap dump
8. Analyze a thread dump
9. GC tuning basics
10. JFR use
11. Memory-leak patterns
12. Thread-pool sizing
13. Connection-pool sizing
14. CPU vs IO bottleneck
15. JIT (high-level)
16. Classloader issues

Security

1. mTLS
2. OAuth authorization-code flow
3. PKCE
4. Token rotation
5. Secrets rotation
6. Threat modeling

7 PCI basics
8 Audit logging
9 OWASP top 10
10 API-abuse prevention

· Future Learning Order

- [ ] Deepen **React** only if the next role is frontend-heavy.
- [ ] Add **DP / graphs** only if target clients ask LeetCode-style questions.
- [ ] Add **AWS / K8s / SRE** if the role is cloud / platform-heavy.
- [ ] Add **JVM internals** if performance / platform questions appear.
- [ ] Add **Cassandra / Search** if the JD names them.
- [ ] Add **architecture governance** for the architect / principal path.

> **KEEP THIS IN ITS LANE**
>
> Nothing here is urgent for a normal senior Java contract round. Pick an area up only when a specific JD or client makes it pay off — otherwise your time belongs to Tracks 1 and 2.